

Scala Fundamentals - Exercise

Data Engineering Training / Independent Lab

Scenario	You are preparing reusable Scala transformation logic before the team moves into Spark. Work with employee and transaction collections and focus on safe, deterministic transformations.

Task 1 - Collections and Transformation Thinking

- Create a List of employee salaries and use map to apply a 10% increase.
- Filter the updated list to retain salaries greater than 80,000.
- Create a Set of departments and demonstrate how duplicate values are handled.
- Create a Map of employee ID -> department and retrieve values safely.

Deliverable: Scala source file and console output.

Task 2 - Pattern Matching

- Given departments Engineering, HR, Finance, Legal and Sales, classify each into a business category using pattern matching.
- Use a default case for an unknown department.
- Apply the classification to a collection using map.

Deliverable: Output showing original department and derived category.

Task 3 - Null Safety with Option

- Create salary data containing Some(value) and None.
- Replace missing salary with 0 using getOrElse.
- Explain in 3-4 lines why Option is preferable to null in data pipelines.

Deliverable: Code, output, and explanation.

Task 4 - Reusable Validation with Traits

- Create a Validator trait with validateSalary(salary:Int): Boolean.
- Create an EmployeeValidator class that uses the trait.
- Test positive, zero and negative salary values.

Deliverable: Trait/class implementation and test output.

Task 5 - Safe Parsing with Try

- Parse List("10","20","abc","40","not_available") into integers without terminating the program.
- Separate valid values from invalid values or apply a documented fallback.
- Print the cleaned collection and the count of invalid inputs.

Deliverable: Code and output.

Challenge: Do not use an unsafe direct `.toInt` on untrusted input.

Task 6 - Mini Pipeline

- Create `Employee(id, name, dept, salary)` as a case class.
- Clean missing/invalid input, filter salary > 70,000, group by department, and calculate employee count, total salary and average salary.
- Keep transformations readable as a pipeline rather than one large imperative block.

Deliverable: `scala_fundamentals_exercise.scala` + output.

Reflection / Interview Check

1. Why do Scala collection transformations map naturally to Spark transformations?
2. What problem does `Option` solve?
3. When would you use `Try` instead of a `try/catch` block?
4. Why can mutable state inside a closure be risky when the same logic later runs in Spark?